

Trabajo práctico: agentes reactivos para refuerzo de taxis

Propósito

Programar y comparar un agente reactivo simple y un agente reactivo basado en modelo. Ambos deben recomendar si una empresa ficticia X debería considerar reforzar con más taxis una zona z durante la hora $h+1$.

El trabajo evalúa reglas condición-acción, percepción, estado interno, dependencia de la historia, causalidad temporal y trazabilidad de las decisiones. No busca construir un predictor de demanda ni optimizar una flota real.

Escenario

La empresa X conoce cuántos taxis propios tiene en la zona z . El entorno genera viajes para varias horas de un lunes y divide sintéticamente esos viajes entre X y otras empresas.

No se conoce el número de vehículos de otras empresas. En su lugar, el entorno simula una proporción de viajes tomados por ellas. Esa proporción es aleatoria, acotada e inversamente proporcional al número de taxis de X:

$$q_{\text{otras}} = \text{clip}(q_{\text{min}} + (q_{\text{max}} - q_{\text{min}}) / (1 + n_X / n_{\text{ref}}) + \epsilon, q_{\text{min}}, q_{\text{max}}), \quad \epsilon \sim N(0, \sigma).$$

Esta relación es una hipótesis didáctica y no describe el comportamiento real de ninguna empresa. Los datos TLC originales solo contienen viajes Yellow Taxi realizados y reportados. La división entre X y otras empresas es completamente sintética.

Preparación

Instalar las dependencias desde la raíz del repositorio:

```
python -m pip install -r requirements.txt
```

Generar un escenario reproducible para la zona TLC 161, tomando la decisión al finalizar la hora 8 y suponiendo 20 taxis de X:

```
python MATERIAL_CURSOS/LABORATORIOS/MOVILIDAD/simulador_entorno_agente.py \  
  
  --zona 161 \  
  
  --hora 8 \  
  
  --taxis-x 20 \  
  
  --horas-historia 3 \  
  
  --semilla 42 \  
  
  --salida-dir escenario_agente
```

El programa crea dos archivos:

Archivo	Uso permitido
escenario_agente/percepciones.csv	Entrada de los agentes; contiene únicamente información hasta h.
escenario_agente/ resultado_h_mas_1.csv	Evaluación posterior; no puede utilizarse para decidir.

Percepción

Cada fila de `percepciones.csv` representa una observación horaria:

Campo	Significado
zona_id	Identificador TLC de la zona.
zona	Nombre de la zona.
hora	Hora cerrada que se está observando.
taxis_x	Taxis de X disponibles al inicio de la hora.
demanda_total	Viajes simulados con pickup en la zona.
tasa_otras_simulada	Proporción sintética sorteada para otras empresas.
viajes_otras	Viajes asignados a otras empresas.
demanda_x	Viajes asignados a X.

Campo	Significado
capacidad_x	Capacidad simplificada de la flota de X.
viajes_atendibles_x	Viajes que X podría atender con esa capacidad.
demanda_no_cubierta_x	Diferencia positiva entre demanda y capacidad.
presion	Cociente entre <code>demanda_x</code> y <code>capacidad_x</code> .

Para este trabajo se supone que cada taxi aporta una unidad de capacidad por hora. La flota se mantiene constante y una recomendación no ejecuta un traslado.

Acciones permitidas

Acción	Interpretación
NO_REFORZAR	La política no encuentra evidencia suficiente para recomendar más taxis.
RECOMENDAR_REFUERZO	Se aconseja que una persona considere reforzar la zona en $h+1$.
ABSTENERSE	La percepción es inválida o no permite aplicar la política con seguridad.

El agente no elige la zona de origen de los taxis ni ejecuta el traslado.

Parte 1: agente reactivo simple

Completar `decidir_reactivo_simple(percepcion)` en `plantilla_agentes_movilidad.py`.

La función debe utilizar exclusivamente la percepción actual y devolver una tupla (`accion`, `motivo`).

Condición	Acción
Faltan datos requeridos, existen valores inválidos o la capacidad es desconocida	ABSTENERSE
<code>presion >= 0.85</code>	RECOMENDAR_REFUERZO
<code>presion < 0.85</code>	NO_REFORZAR

No está permitido utilizar variables globales mutables, observaciones anteriores ni el archivo de evaluación futura.

Parte 2: agente reactivo basado en modelo

Completar `actualizar_estado(estado_anterior, percepcion)` y `decidir_reactivo_modelo(estado_actual)`.

El estado interno mínimo es:

```
{
    "percepcion_valida": False,
    "racha_presion_alta": 0,
    "presion_anterior": None,
    "ultima_accion": None,
}
```

La actualización debe incrementar `racha_presion_alta` cuando `presion >= 0.85` y reiniciarla cuando la presión sea menor. Los datos inválidos deben dejar el estado en una condición que produzca `ABSTENERSE`.

Estado actualizado	Acción
Percepción inválida	ABSTENERSE
Dos o más horas consecutivas con presión alta	RECOMENDAR_REFUERZO
Cualquier otro estado válido	NO_REFORZAR

Este agente sigue siendo reactivo: actualiza un resumen de la historia y aplica reglas al estado actual. No genera sucesores, no busca caminos y no planifica traslados.

Parte 3: bitácora y comparación

Completar `procesar_secuencia(percepciones)` para ejecutar ambos agentes en orden temporal. La salida debe contener como mínimo:

Campo	Contenido
<code>hora</code>	Hora observada.
<code>presion</code>	Presión percibida.
<code>racha_presion_alta</code>	Estado persistente después de actualizar.
<code>accion_simple</code>	Acción del agente reactivo simple.

Campo	Contenido
<code>motivo_simple</code>	Justificación de la regla aplicada.
<code>accion_modelo</code>	Acción del agente basado en modelo.
<code>motivo_modelo</code>	Justificación basada en el estado.

Responder brevemente:

1. ¿En qué situaciones ambos agentes producen la misma acción?
2. ¿Cuándo reaccionan de forma diferente?
3. ¿Por qué el segundo agente está basado en modelo aunque no planifique?
4. ¿Qué representa `tasa_otras_simulada` y qué no permite afirmar?
5. ¿Por qué `resultado_h_mas_1.csv` no puede formar parte de la percepción?

Pruebas obligatorias

Implementar al menos estos tres casos:

Caso	Comprobación esperada
Presión baja	Ambos agentes devuelven <code>NO_REFORZAR</code> .
Primera hora con presión alta	El simple recomienda refuerzo y el basado en modelo todavía no.
Segunda hora consecutiva con presión alta	Ambos recomiendan refuerzo.

Agregar una prueba decisiva con dos historias distintas que terminen en la misma percepción. El agente reactivo simple debe producir la misma acción en ambos casos. El agente basado en modelo puede producir acciones diferentes porque su estado anterior es distinto.

También se debe comprobar que ninguna función de decisión recibe o consulta datos de `h+1`.

PEAS mínimo

La entrega debe identificar:

Elemento	Contenido esperado
Performance	Recomendaciones coherentes con las reglas, ausencia de fuga temporal, abstención ante datos inválidos y trazabilidad.
Environment	Secuencia simulada zona-hora, demanda TLC transformada, flota sintética de X, otras

Elemento	Contenido esperado
	empresas sintéticas y responsable humano.
Actuators	Mensajes <code>NO_REFORZAR</code> , <code>RECOMENDAR_REFUERZO</code> y <code>ABSTENERSE</code> .
Sensors	Lectura lógica de <code>percepciones.csv</code> ; no es un sensor conectado en tiempo real.

Entrega

1. `agentes_movilidad.py` con las funciones completas.
2. `test_agentes_movilidad.py` con las pruebas solicitadas.
3. `bitacora_agentes.csv` con la comparación sobre un escenario reproducible.
4. Un informe breve en Markdown con las respuestas, el PEAS y las limitaciones.

Evaluación

Criterio	Puntaje
Agente reactivo simple y validación de entradas	2
Actualización correcta del estado interno	2
Política del agente basado en modelo	2
Pruebas, bitácora y demostración de dependencia histórica	2
PEAS, causalidad temporal y discusión de limitaciones	2

Limitaciones que deben reconocerse

- Los pickups TLC son actividad Yellow Taxi realizada y reportada, no demanda total ni solicitudes no atendidas.
- X y las otras empresas son entidades ficticias creadas para el ejercicio.
- La relación inversa entre flota de X y participación externa no ha sido estimada con datos reales.
- Una unidad de capacidad por taxi-hora es una simplificación.
- La distancia entre centroides no determina duración ni disponibilidad.
- `RECOMENDAR_REFUERZO` es un mensaje para revisión humana, no una orden ni un traslado ejecutado.